

Large Language Models (LLMs): From Basics to Advanced

Authored by: Siddharth Vidyarthi

Edition date: 2026-03-30

Copyright & license

This e-book is original educational material authored by Siddharth Vidyarthi. You may share it internally for learning and interview preparation. If you plan to republish publicly, keep attribution and ensure you have rights for any code/data you add.

How to use this book

- If you're new: read Chapters 1–6 in order, then jump to Chapter 10 (RAG) and Chapter 15 (Evaluation).
 - If you're prepping for interviews: skim the Notes at the end of each chapter, then go straight to **Chapter 20: 120 Real Interview Questions & Answers**.
 - If you learn by doing: start with **Chapter 12–14 coding labs**, then attempt **Chapter 21: Medium–Advanced Practice Questions**.
-

Table of contents

1. What an LLM is (and isn't)
2. The Transformer, attention, and why it scaled
3. Tokenization and subword vocabularies
4. Pretraining objectives and scaling intuitions
5. Instruction tuning and RLHF (alignment basics)
6. Inference: decoding, sampling, and controllability
7. Prompt engineering: patterns that survive production
8. Fine-tuning: full vs PEFT (LoRA/QLoRA)

9. Embeddings, vector search, and ANN indexes
 10. Retrieval-Augmented Generation (RAG): design and pitfalls
 11. Tool use / function calling and agentic workflows
 12. Coding Lab 1 — Build a tiny Transformer (PyTorch)
 13. Coding Lab 2 — Fine-tune with LoRA (Transformers + PEFT)
 14. Coding Lab 3 — Minimal RAG pipeline (Python)
 15. Evaluation: offline metrics, human eval, and HELM-style thinking
 16. LLMops: latency, cost, caching, and observability
 17. Safety & security: OWASP risks, prompt injection, data leakage
 18. Case studies (real-world deployments)
 19. Cheat sheets + notes (printable)
 20. 120 real interview questions & answers
 21. Medium–advanced practice questions (with solutions)
 22. Appendix: recommended papers, docs, and study plan
-

Chapter 1 — What an LLM is (and isn't)

A **Large Language Model (LLM)** is a neural network trained on large text corpora to predict the next token (or fill missing tokens) and, as a result, learn statistical structure of language. In modern systems, the same base model is adapted to follow instructions, call tools, write code, and reason over long contexts.

Mental model

- Pretraining teaches broad language competence.
- Adaptation (instruction tuning / RLHF / preference optimization) teaches *useful behavior*.
- Retrieval and tools teach *grounding* and *capabilities beyond parameters*.

Key terms

- **Token:** the atomic unit the model sees (often a subword).
- **Context window:** how many tokens the model can attend to at once.
- **Hallucination:** confident generation not supported by evidence.

Notes (Chapter 1)

- Think “probabilistic text engine + adapters + product constraints”, not “magic intelligence”.
 - Most reliability wins come from *systems design* (retrieval, tools, evals), not just bigger models.
-

Chapter 2 — The Transformer, attention, and why it scaled

Transformers replaced recurrence with attention and parallel computation, enabling efficient scaling. The original Transformer paper (“Attention Is All You Need”) introduced the encoder–decoder Transformer built from multi-head self-attention and feed-forward blocks.

Core equations (conceptual)

Self-attention takes queries, keys, values (Q, K, V) and computes weighted combinations of V based on similarity of Q and K.

Why it scaled

- Parallelizable training (no sequential recurrence bottleneck).
- Strong inductive bias for global dependency modeling.
- Works well with massive datasets and compute.

Practical takeaway

If you can explain “why attention is $O(n^2)$ ” and why long-context needs algorithmic tricks (FlashAttention, sparsity, chunking), you’re already ahead in interviews.

Notes (Chapter 2)

- Attention is differentiable content-addressing.
 - Long context is not free: memory/time grow quickly; production needs careful trade-offs.
-

Chapter 3 — Tokenization and subword vocabularies

LLMs operate on tokens, not characters or words. Tokenizers (BPE, Unigram, WordPiece) compress text into subword units to balance vocabulary size and coverage.

What interviewers love

- Why tokenization matters for cost and latency (tokens are billing + compute units).
- Why multilingual and code tokenization differs.
- Why byte-level BPE reduces “unknown token” problems.

Notes (Chapter 3)

- Tokenization artifacts explain many weird failures (spacing, punctuation, URLs).
-

Chapter 4 — Pretraining objectives and scaling intuitions

Common pretraining objectives:

- **Causal Language Modeling (CLM)**: predict next token (GPT-style).
- **Masked Language Modeling (MLM)**: predict masked tokens (BERT-style).
- **Denoising seq2seq**: reconstruct corrupted text (T5-style).

Scaling tends to improve generalization when you increase data, parameters, and compute—until you hit data quality limits, optimization issues, or deployment constraints.

Notes (Chapter 4)

- Data quality often beats data quantity at later stages.
 - Memorization risks rise with scale; privacy and governance become mandatory.
-

Chapter 5 — Instruction tuning and RLHF (alignment basics)

Instruction tuning trains the model on (instruction, response) pairs so it follows user intent. RLHF (Reinforcement Learning from Human Feedback) typically adds a reward model and RL step (e.g., PPO) so outputs better match human preferences.

A standard RLHF-ish pipeline

1. Supervised fine-tuning on demonstrations.
2. Train a reward model on ranked outputs.
3. Optimize the policy model against the reward model.

Notes (Chapter 5)

- Alignment is not a switch; it's a continuous process plus strong product guardrails.
-

Chapter 6 — Inference: decoding, sampling, and controllability

Decoding strategies shape output quality and safety.

- Greedy: fast, can be dull.
- Beam search: good for translation; can be repetitive for chat.
- Sampling: temperature, top-k, top-p (nucleus sampling).

Notes (Chapter 6)

- “Better prompts” often mean “better decoding + better constraints”.
-

Chapter 7 — Prompt engineering: patterns that survive production

Prompting is interface design.

Reliable prompt patterns

- Role + goal + constraints + output schema.
- Provide examples (few-shot) when format matters.
- Ask for citations to provided context (not to the open web) when you’re doing RAG.

Anti-patterns

- Overlong prompts that hide the real requirement.
- Using the model as a database.

Notes (Chapter 7)

- Prompts should be versioned, tested, and treated like code.
-

Chapter 8 — Fine-tuning: full vs PEFT (LoRA/QLoRA)

Fine-tuning adapts a base model to a domain or style. Full fine-tuning updates all parameters; PEFT updates a small subset (adapters), reducing compute and storage.

LoRA intuition

LoRA freezes base weights and learns low-rank updates injected into certain linear layers (often attention projections). It's one of the most common PEFT approaches.

Notes (Chapter 8)

- PEFT is usually the default unless you have strong reasons (and budget) for full tuning.
-

Chapter 9 — Embeddings, vector search, and ANN indexes

Embeddings map text into vectors so semantically similar text is close in vector space. Vector similarity search retrieves top-k relevant chunks via approximate nearest neighbor (ANN) indexing.

Notes (Chapter 9)

- Retrieval quality depends more on chunking + metadata filters than on “the fanciest index”.
-

Chapter 10 — Retrieval-Augmented Generation (RAG): design and pitfalls

RAG combines retrieval with generation so responses are grounded in external context, and knowledge can be updated without retraining.

A production RAG checklist

- Define corpus + freshness needs.
- Chunking: structure-aware chunking (headings, tables) beats naive fixed windows.
- Index: embeddings + ANN search + metadata filters.
- Prompt: force model to answer from retrieved context; return “not found” otherwise.
- Evaluation: measure retrieval hit rate, attribution accuracy, and refusal behavior.

Failure modes

- Wrong retrieval (semantic drift).
- Over-reliance on low-quality docs.
- Prompt injection via retrieved content.

Notes (Chapter 10)

- RAG is an *information system* problem first, LLM problem second.
-

Chapter 11 — Tool use / function calling and agentic workflows

Tool use lets an LLM call deterministic functions (DB queries, calculators, search, ticket creation) with structured arguments.

What “good” tool use needs

- Strict JSON schema validation.
- Idempotency and retries.
- Audit logs, user consent for sensitive actions.

Notes (Chapter 11)

- Don’t let the model “free-write” tool arguments; enforce schemas.
-

Chapter 12 — Coding Lab 1:

Build a tiny Transformer (PyTorch)

This lab is for understanding, not performance.

```
import math
import torch
import torch.nn as nn

class MultiHeadSelfAttention(nn.Module):
    def __init__(self, d_model: int, n_heads: int):
        super().__init__()
        assert d_model % n_heads == 0
        self.d_model = d_model
        self.n_heads = n_heads
        self.d_head = d_model // n_heads
        self.qkv = nn.Linear(d_model, 3 * d_model)
        self.out = nn.Linear(d_model, d_model)

    def forward(self, x, attn_mask=None):
        B, T, C = x.shape
        qkv = self.qkv(x)
        q, k, v = qkv.chunk(3, dim=-1)

        def reshape(t):
            return t.view(B, T, self.n_heads, self.d_head).transpose(1, 2)

        q, k, v = map(reshape, (q, k, v))
        scores = (q @ k.transpose(-2, -1)) / math.sqrt(self.d_head)
        if attn_mask is not None:
            scores = scores.masked_fill(attn_mask == 0, float('-inf'))
        w = torch.softmax(scores, dim=-1)
        y = w @ v
        y = y.transpose(1, 2).contiguous().view(B, T, C)
```

```

    return self.out(y)

class TransformerBlock(nn.Module):
    def __init__(self, d_model: int, n_heads: int, d_ff: int):
        super().__init__()
        self.ln1 = nn.LayerNorm(d_model)
        self.attn = MultiHeadSelfAttention(d_model, n_heads)
        self.ln2 = nn.LayerNorm(d_model)
        self.ff = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.GELU(),
            nn.Linear(d_ff, d_model),
        )

    def forward(self, x, attn_mask=None):
        x = x + self.attn(self.ln1(x), attn_mask=attn_mask)
        x = x + self.ff(self.ln2(x))
        return x

```

Exercises

1. Add causal masking for autoregressive decoding.
2. Add dropout and compare training stability.
3. Implement KV caching for fast decoding.

Notes (Chapter 12)

- If you can implement attention from scratch, debugging real training code becomes much easier.
-

Chapter 13 — Coding Lab 2: Fine-tune with LoRA (Transformers + PEFT)

This lab shows the standard workflow. Use a small model locally (e.g., 0.5B–3B) for practice.

```
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForCausalLM, TrainingArguments
from transformers import DataCollatorForLanguageModeling, Trainer
from peft import LoraConfig, get_peft_model, TaskType

model_name = "distilgpt2"

dataset = load_dataset("imdb", split="train[:1%]")

tokenizer = AutoTokenizer.from_pretrained(model_name)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

def tok(batch):
    return tokenizer(batch["text"], truncation=True, max_length=256)

ds = dataset.map(tok, batched=True, remove_columns=["text"])

base = AutoModelForCausalLM.from_pretrained(model_name)

lora = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    r=8,
    lora_alpha=16,
    lora_dropout=0.05,
    target_modules=["c_attn"],
)
```

```

model = get_peft_model(base, lora)

args = TrainingArguments(
    output_dir="./lora-out",
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4,
    learning_rate=2e-4,
    num_train_epochs=1,
    logging_steps=10,
    save_steps=100,
)

collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False)

trainer = Trainer(
    model=model,
    args=args,
    train_dataset=ds,
    data_collator=collator,
)

trainer.train()

```

Exercises

1. Replace IMDB with your own domain dataset (company docs, tickets, FAQs).
2. Compare: instruction-tuned vs base model fine-tuning.
3. Add eval: perplexity + a small human preference test.

Notes (Chapter 13)

- Most “fine-tuning failed” issues are actually data formatting issues.
-

Chapter 14 — Coding Lab 3:

Minimal RAG pipeline (Python)

Below is a vendor-neutral conceptual pipeline.

```
import numpy as np

def embed(texts):
    # Replace with your embedding model call
    raise NotImplementedError

def cosine(a, b):
    a = a / (np.linalg.norm(a) + 1e-9)
    b = b / (np.linalg.norm(b) + 1e-9)
    return float(np.dot(a, b))

class TinyVectorIndex:
    def __init__(self):
        self.texts = []
        self.vecs = []

    def add(self, texts):
        v = embed(texts)
        self.texts.extend(texts)
        self.vecs.extend(v)

    def search(self, query, k=5):
        qv = embed([query])[0]
        scores = [(cosine(qv, v), t) for v, t in zip(self.vecs, self.texts)]
        scores.sort(reverse=True, key=lambda x: x[0])
        return scores[:k]

    def build_prompt(query, retrieved):
        ctx = ""

        ".join([f"- {t}" for _, t in retrieved])
```

```
return (  
    "You are a helpful assistant.  
"  
    "Answer using ONLY the context. If the context is insufficient, say: I don  
"  
    f"Question: {query}  
"  
    f"Context:  
{ctx}  
"  
    "Answer:"  
)
```

Exercises

1. Add chunking + metadata (doc_id, section, timestamp).
2. Add guardrails against prompt injection in retrieved text.
3. Add citations by returning doc_id + span offsets.

Notes (Chapter 14)

- RAG success is mostly about *document engineering*.

Chapter 15 — Evaluation: offline metrics, human eval, and HELM-style thinking

Evaluation is where serious teams differentiate.

A practical evaluation stack

- Unit tests: formatting, schema compliance, refusal policy.
- Retrieval tests: known-answer questions with expected citations.
- Model grading: LLM-as-judge *with calibration*.
- Human eval: sampling + rubrics for correctness, helpfulness, safety.

Notes (Chapter 15)

- If you can't measure it, you can't ship it.
-

Chapter 16 — LLMOps: latency, cost, caching, and observability

Production LLM systems need to be engineered like distributed systems.

Key practices

- Prompt + context caching.
- Streaming responses.
- Rate limits, backpressure, retries.
- Tracing: prompt, retrieved docs, tool calls, latency breakdown.

Notes (Chapter 16)

- Observability is the difference between “cool demo” and “reliable product”.
-

Chapter 17 — Safety & security: OWASP risks, prompt injection, data leakage

LLM apps have unique security risks (prompt injection, data exfiltration, insecure tool use, etc.). Treat them as a new application security domain.

Practical defenses

- Treat all model inputs as untrusted (including retrieved documents).
- Sandbox tools; never give the model raw credentials.
- Output encoding + allowlists for downstream systems.

Notes (Chapter 17)

- Secure-by-design beats “prompt-only” defenses.
-

Chapter 18 — Case studies (real-world deployments)

This chapter summarizes widely discussed public deployments to extract patterns.

Case study A: Internal knowledge assistant (wealth management)

Pattern: RAG over a large internal document corpus + strict access controls + evals before rollout.

Case study B: Language learning features with LLMs

Pattern: LLM as a *tutor* for explanations and roleplay, with product constraints to keep output on-task.

Case study C: Developer copilots

Pattern: Context (repo, diffs) + tight IDE integration + telemetry + policy constraints.

What you should learn

- Adopt a "narrow reliable" use case first.
 - Instrument everything.
 - Evals are a product feature.
-

Chapter 19 — Cheat sheets + notes (printable)

Transformer cheat sheet

- Block = LN → Attention → residual → LN → FFN → residual.
- Attention = similarity(Q,K) → weights → weighted sum(V).

RAG cheat sheet

- Bad RAG: huge chunks, no metadata filters, no eval.
- Good RAG: structure-aware chunking, filters, citations, refusal.

Fine-tuning cheat sheet

- Use PEFT by default.
 - Always validate data formatting with a tiny run.
-

Chapter 20 — 120 real interview questions & answers

These are **original** questions and answers written for realistic ML/LLM engineering interviews (screening to onsite). Answers are short on purpose—what matters is your ability to expand with trade-offs.

Fundamentals (1–25)

1. **Q:** What is an LLM? **A:** A neural model trained on large text to predict tokens; capabilities emerge from pretraining + alignment.
2. **Q:** What is a token? **A:** A subword unit produced by a tokenizer; it's the model's basic I/O unit.
3. **Q:** Why Transformers over RNNs? **A:** Better parallelism and long-range dependency modeling.
4. **Q:** What is self-attention? **A:** A mechanism to compute context-aware representations by weighting other tokens.
5. **Q:** Why multi-head attention? **A:** Multiple subspaces capture diverse relationships.
6. **Q:** What is positional encoding? **A:** A way to inject token order information into attention.
7. **Q:** Explain causal masking. **A:** Prevents attending to future tokens during autoregressive generation.
8. **Q:** What is CLM vs MLM? **A:** CLM predicts next token; MLM predicts masked tokens.
9. **Q:** What is a context window? **A:** Maximum tokens the model can attend to in one pass.
10. **Q:** What is temperature? **A:** Sampling parameter controlling randomness.
11. **Q:** Top-k vs top-p? **A:** Top-k picks from k tokens; top-p picks smallest set with cumulative prob p.
12. **Q:** What is repetition penalty? **A:** Heuristic discouraging repeating tokens/phrases.
13. **Q:** What is hallucination? **A:** Confident generation not supported by data/context.

14. **Q:** How do you reduce hallucination? **A:** Retrieval grounding, constrained prompting, better evals, refusal.
15. **Q:** What is instruction tuning? **A:** Supervised training on instruction–response pairs.
16. **Q:** What is RLHF? **A:** Preference optimization using human feedback via reward modeling + RL.
17. **Q:** What is a reward model? **A:** Model that scores outputs; used to optimize policy.
18. **Q:** What is PPO in RLHF? **A:** A stable policy-gradient method used to update the model.
19. **Q:** What is “alignment” in practice? **A:** Making outputs match user intent and safety constraints.
20. **Q:** Why do smaller aligned models sometimes beat bigger base models? **A:** Behavior tuning can dominate raw capability.
21. **Q:** What is perplexity? **A:** Exponentiated average negative log-likelihood.
22. **Q:** Why is perplexity insufficient for chat quality? **A:** It doesn’t capture instruction following or helpfulness.
23. **Q:** What is parameter count vs context length trade-off? **A:** Both impact capability; long context raises memory/time.
24. **Q:** What’s KV cache? **A:** Storing key/value tensors for past tokens to speed decoding.
25. **Q:** What is FlashAttention conceptually? **A:** IO-aware attention computation to reduce memory traffic.

Data & training (26–50)

26. **Q:** What data issues break fine-tuning? **A:** Bad formatting, mixed schemas, label leakage, duplicates.
27. **Q:** What is catastrophic forgetting? **A:** Overwriting general capabilities when fine-tuning narrowly.
28. **Q:** How to mitigate forgetting? **A:** Smaller LR, adapters, mix-in general data.
29. **Q:** What is LoRA? **A:** Low-rank adapters that learn updates while freezing base weights.
30. **Q:** Why PEFT? **A:** Cheaper training, smaller artifacts, easier multi-tasking.
31. **Q:** What is QLoRA? **A:** LoRA applied on quantized weights to save memory.

- 32. **Q:** When would you full fine-tune? **A:** When domain shift is large and you can afford it.
- 33. **Q:** What is SFT? **A:** Supervised fine-tuning.
- 34. **Q:** What is preference data? **A:** Ranked comparisons of model outputs.
- 35. **Q:** How do you build an instruction dataset? **A:** Collect tasks, write prompts, create high-quality answers, validate.
- 36. **Q:** How do you deduplicate? **A:** Hashing, n-gram similarity, embedding similarity.
- 37. **Q:** What is data contamination? **A:** Eval items leaking into training.
- 38. **Q:** Why is contamination dangerous? **A:** Inflated benchmark scores.
- 39. **Q:** What is gradient checkpointing? **A:** Trading compute for memory by recomputing activations.
- 40. **Q:** What is mixed precision? **A:** Using fp16/bf16 for speed/memory.
- 41. **Q:** What is bf16 vs fp16 difference? **A:** bf16 has larger exponent range, often more stable.
- 42. **Q:** What is sequence packing? **A:** Combining short examples into one sequence for efficiency.
- 43. **Q:** What is loss masking in chat SFT? **A:** Only compute loss on assistant tokens.
- 44. **Q:** Why does chat formatting matter? **A:** Token boundaries define what's learned and predicted.
- 45. **Q:** What are eval splits best practices? **A:** Time-based splits, holdout domains, leakage checks.
- 46. **Q:** What is curriculum learning? **A:** Training from easy to hard examples.
- 47. **Q:** What is RLHF failure mode? **A:** Reward hacking and over-optimization.
- 48. **Q:** How do you detect reward hacking? **A:** Adversarial prompts, human audits, reward-quality divergence.
- 49. **Q:** What is "alignment tax"? **A:** Capability reduction due to constraints/tuning.
- 50. **Q:** How do you tune refusal rates? **A:** Policy prompts, classifier gates, sampling, eval targets.

RAG & search (51–75)

- 51. **Q:** What problem does RAG solve? **A:** Freshness, grounding, citations, domain knowledge without retrain.
- 52. **Q:** What is embedding? **A:** Dense vector representation capturing semantics.
- 53. **Q:** Why chunk documents? **A:** Retrieval works on chunks; too big reduces precision.
- 54. **Q:** Best chunk size? **A:** Depends; start 300–800 tokens and evaluate.
- 55. **Q:** Why add overlap? **A:** Preserve cross-boundary context.
- 56. **Q:** What is ANN? **A:** Approximate nearest neighbor search for speed.
- 57. **Q:** Common ANN indexes? **A:** HNSW, IVF, PQ (implementation-dependent).
- 58. **Q:** What is FAISS? **A:** A library for efficient similarity search and clustering.
- 59. **Q:** What is hybrid search? **A:** Combine lexical (BM25) + vector search.
- 60. **Q:** When does BM25 beat embeddings? **A:** Exact terms, identifiers, rare tokens.
- 61. **Q:** What is reranking? **A:** Re-scoring retrieved docs with a cross-encoder or LLM.
- 62. **Q:** How do you evaluate retrieval? **A:** Recall@k, MRR, nDCG, answer accuracy with citations.
- 63. **Q:** What is “citation accuracy”? **A:** Whether claims are supported by provided sources.
- 64. **Q:** How do you prevent RAG prompt injection? **A:** Strip instructions, isolate context, enforce policies, content filters.
- 65. **Q:** What is metadata filtering? **A:** Restrict retrieval by tags (team, region, access level).
- 66. **Q:** What is freshness strategy? **A:** Incremental ingestion, time decay, re-embedding triggers.
- 67. **Q:** How to handle tables? **A:** Structure-aware extraction; sometimes table-to-text.
- 68. **Q:** How to handle PDFs? **A:** Layout-aware parsing; store coordinates and headings.
- 69. **Q:** What’s a vector DB? **A:** Storage + indexing + query for embeddings with metadata.

70. **Q:** Do you always need a vector DB? **A:** No; small corpora can use in-memory or pgvector.
71. **Q:** What is grounding? **A:** Using retrieved evidence to answer.
72. **Q:** What is “stuffing” context? **A:** Dumping too many docs; hurts focus and cost.
73. **Q:** How do you choose k? **A:** Optimize trade-off between recall and token budget.
74. **Q:** What’s query rewriting? **A:** Transform user question into retrieval-friendly queries.
75. **Q:** What’s multi-hop retrieval? **A:** Retrieve iteratively based on intermediate answers.

Tool use & agents (76–95)

76. **Q:** What is function calling? **A:** Structured API for the model to request tool invocations.
77. **Q:** Why strict schemas? **A:** Prevents injection into downstream systems.
78. **Q:** What is tool hallucination? **A:** Model invents tools/arguments.
79. **Q:** Fix tool hallucination? **A:** Restrict tool list, examples, validation, retries.
80. **Q:** What is an agent? **A:** LLM loop that plans, acts via tools, and observes results.
81. **Q:** When are agents overkill? **A:** Simple single-step tasks; use direct calls.
82. **Q:** What is a planner–executor pattern? **A:** Separate planning from action to reduce errors.
83. **Q:** What is reflection? **A:** Model critiques and improves its output.
84. **Q:** Main risk of reflection? **A:** More tokens and sometimes more confident wrongness.
85. **Q:** How to bound agent loops? **A:** Step limits, timeouts, budgets, termination rules.
86. **Q:** What is idempotency? **A:** Safe retries without duplicated side effects.
87. **Q:** How to secure tools? **A:** Least privilege, sandboxing, human approval for high-impact actions.
88. **Q:** What is data exfiltration via tools? **A:** Model leaks secrets by calling tools.

- 89. **Q:** How to prevent exfiltration? **A:** Policy engine, redaction, allowlists, monitoring.
- 90. **Q:** What is structured output? **A:** Guaranteed JSON-like format for parsing.
- 91. **Q:** What is deterministic decoding? **A:** Low temperature / greedy to reduce variance.
- 92. **Q:** When do you want randomness? **A:** Brainstorming, creative writing, diverse candidates.
- 93. **Q:** What is chain-of-thought? **A:** Intermediate reasoning text (not always safe to expose).
- 94. **Q:** Why not always reveal reasoning? **A:** Can leak sensitive info and encourage prompt attacks.
- 95. **Q:** How do you test an agent? **A:** Simulated environments, golden traces, adversarial tasks.

Production, safety, and debugging (96–120)

- 96. **Q:** What is prompt injection? **A:** Malicious instructions embedded in user/context to hijack behavior.
- 97. **Q:** What is jailbreak? **A:** Attempts to bypass safety constraints.
- 98. **Q:** How do you reduce jailbreak risk? **A:** Defense-in-depth: policy, filters, tool sandbox, monitoring.
- 99. **Q:** What's OWASP LLM Top 10 for? **A:** Catalog of common LLM app vulnerabilities.
- 100. **Q:** What is model DoS? **A:** Token flooding, expensive prompts, tool abuse.
- 101. **Q:** How to prevent DoS? **A:** Rate limits, token caps, caching, early exits.
- 102. **Q:** How to reduce latency? **A:** Smaller models, quantization, batching, KV cache, caching retrieval.
- 103. **Q:** How to reduce cost? **A:** Cache, compress context, use smaller models for easy cases.
- 104. **Q:** What is routing? **A:** Choose model/tool path based on query type.
- 105. **Q:** What is A/B testing for LLMs? **A:** Compare prompts/models with online metrics and safety checks.
- 106. **Q:** What online metrics matter? **A:** CSAT, deflection, latency, escalation rate, safety incidents.

107. **Q:** Why is offline eval not enough? **A:** Real users behave differently; distribution shifts.
108. **Q:** How to monitor quality drift? **A:** Periodic evals, canary prompts, feedback loops.
109. **Q:** What is telemetry privacy risk? **A:** Logging sensitive prompts.
110. **Q:** How to log safely? **A:** Redaction, hashing, sampling, access controls.
111. **Q:** What is data residency? **A:** Storing/processing data in specific regions.
112. **Q:** What is PHI/PII handling? **A:** Detect and redact, minimize retention.
113. **Q:** What is retrieval leakage? **A:** Returning private docs to unauthorized users.
114. **Q:** How to prevent leakage? **A:** AuthZ at retrieval time; per-user indexes or filters.
115. **Q:** Why “citations” help? **A:** Auditable grounding and user trust.
116. **Q:** What is “not found” behavior? **A:** Model must refuse when evidence missing.
117. **Q:** How to debug hallucinations? **A:** Inspect retrieved context, prompt, decoding, and constraints.
118. **Q:** What is prompt regression testing? **A:** Golden prompts with expected outputs.
119. **Q:** What’s the best first LLM project? **A:** Internal FAQ/RAG assistant with tight scope.
120. **Q:** How do you explain LLM risk to executives? **A:** Focus on security, privacy, quality metrics, and governance.
-

Chapter 21 — Medium–advanced practice questions (with solutions)

Practice set A (conceptual + design)

1. Design a RAG assistant for a 5M-document corpus with access control. Provide: chunking, indexing, authZ, eval, monitoring.
2. Propose an eval plan for a customer-support LLM that must never fabricate policy.
3. Compare LoRA vs full fine-tune for a multilingual support bot; include cost and operational complexity.
4. Design prompt-injection defenses for: (a) user prompt, (b) retrieved docs, (c) tool outputs.
5. Propose a routing system: small model for simple queries, large model for complex, with confidence estimation.

Practice set B (coding)

6. Implement top-p sampling from logits.
7. Implement a causal attention mask for variable-length batches.
8. Write a chunker that respects headings and keeps tables intact.
9. Implement an embedding cache keyed by (doc_id, chunk_hash).
10. Implement a minimal evaluator that checks whether an answer quotes at least one retrieved span.

Suggested solutions (high-level)

- For #1, enforce authZ at retrieval time and keep audit logs; evaluate citation accuracy.
 - For #6, sort probabilities, take prefix up to p, renormalize, sample.
 - For #10, parse citations or quoted spans; fail closed if missing.
-

Chapter 22 — Appendix: recommended papers, docs, and study plan

Core papers / resources

- Transformer: *Attention Is All You Need* (Vaswani et al., 2017) — <https://arxiv.org/abs/1706.03762>
- RLHF / instruction-following: *Training language models to follow instructions with human feedback* (Ouyang et al., 2022) — <https://arxiv.org/abs/2203.02155>
- LoRA: *Low-Rank Adaptation of Large Language Models* (Hu et al., 2021) — <https://arxiv.org/abs/2106.09685>
- RAG: *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks* (Lewis et al., 2020) — <https://arxiv.org/abs/2005.11401>
- HELM evaluation: *Holistic Evaluation of Language Models* (Liang et al., 2022) — <https://arxiv.org/abs/2211.09110>
- FlashAttention: *Fast and Memory-Efficient Exact Attention with IO-Awareness* (Dao et al., 2022) — <https://arxiv.org/abs/2205.14135>
- OWASP LLM Top 10: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- Hugging Face Transformers (fine-tuning CLM): https://huggingface.co/docs/transformers/en/tasks/language_modeling
- Hugging Face PEFT (LoRA): https://huggingface.co/docs/peft/main/conceptual_guides/lora

4-week study plan (engineering track)

- Week 1: Transformers + tokenization; implement attention + masking (Lab 1).
- Week 2: Fine-tuning + PEFT; train small LoRA models (Lab 2).
- Week 3: RAG systems + vector search; build minimal RAG (Lab 3).

- Week 4: Evals + LLMOps + security; do red-team prompts + regression tests.

End of book.